

项目管理基础

软件工程与计算 II · 模块一 · 单元 2

当一次部署遗漏了 1 台服务器，4400 万美元在 45 分钟内蒸发

课程	软件工程与计算 II
专业	软件工程
讲师	刘钦

今日学习路径

① 真实案例

一次部署事故
损失 4400 万美元

② 项目管理

项目·团队·激励
风险管理·AI变革

③ 软件质量保障

评审·测试·审计
AI时代质量新挑战

④ 配置管理

SCM·DevOps·GitOps
CI/CD·IaC·LLMOps

⑤ 总结



知识框架
作业·预告

三个核心问题，今天结束时你应该能回答：

1. 软件团队的三种结构有何优劣？**什么情况下用哪种？**
2. 软件质量保障（SQA）包含哪些活动？**评审和测试的分工是什么？**
3. 配置管理的四大活动是什么？**Git 分支策略**如何防止代码混乱？

PART 1 · 从一次部署事故开始

Knight Capital · 45 分钟 · 4.4 亿美元消失

时间	事件
2012-08-01 09:30	美股开盘，Knight Capital 的新交易系统上线
09:30-10:15	系统异常，以错误价格持续买卖股票， 45 分钟内亏损 4.4 亿美元
调查结论	8 台交易服务器中， 1 台未被更新 ，仍运行含有旧"Power Peg"代码的旧版本
根本原因	部署流程无检查清单 ，配置状态未验证，旧代码功能未彻底禁用
最终结局	Knight Capital 被迫出售，实际上 公司消失

不是算法错误，不是网络故障——是一次配置管理的彻底失效

8 台服务器中有 7 台是"新软件"，1 台是"旧软件"——系统无法自动发现这一不一致

一次遗漏的部署，如何击垮一家公司？

项目管理缺陷（进度压力下跳过验证）

"赶在开盘前部署完成，不需要逐台确认"

↓ 配置控制缺失

部署清单不存在或未执行

← 配置控制缺失 (No Change Control)

第 8 台服务器状态未验证

← 状态报告缺失 (No Status Accounting)

旧功能标志未统一清理

← 配置项管理不完整

上线前无回滚预案

← 变更影响分析缺失

↓ 不一致的配置状态在真实交易中触发旧逻辑

旧"Power Peg"模块持续以市价买入 → 积累 70 亿美元错误头寸

↓ 45 分钟

亏损 4.4 亿美元 → 公司被迫出售

停下来想一想

这次事故最应该在哪个环节被拦截？
是技术问题，还是管理问题，还是两者都有？

和旁边的同学交流 1 分钟，然后分享你的想法

参考思路：

- 部署前是否有**逐台验证**的 Checklist?
- 是否有自动化工具确认**所有节点版本一致**?
- 上线前是否进行了**变更影响评估**（旧功能是否完全禁用）？
- 是否有**自动熔断机制**：亏损超过阈值时立即停止交易？

💡 这些都是**软件配置管理和项目风险管理**的标准实践——
而不是高深的技术问题

这个案例告诉我们什么？

核心问题	案例映射
配置管理为何是工程纪律？	1 台服务器的配置不一致 = 公司消失
项目管理中的进度压力如何制造风险？	"赶进度"跳过验证步骤，代价是灾难性的
变更控制流程为何不能省略？	没有上线前状态审计，错误在真实环境中才被发现

💡 **关键洞察：** Knight Capital 的工程师们技术能力很强——
失效的不是技术，而是**过程与管理**
(SEC Investigation Report, 2013)

PART 2 · 项目管理基础

什么是"项目"?

项目是为创建**唯一**产品、服务或成果而进行的**临时性**工作。

——PMBOK（项目管理知识体系指南）

要素	说明	与"日常运营"的区别
临时性	有明确的开始和结束时间	运营是持续的、无终止日期的
唯一性	产出物是独特的，不完全重复	运营是标准化、重复的流程
渐进明细	随进展逐步明确细节	运营流程通常是预先固化的

项目管理的目标：在铁三角约束下交付质量



项目的经典定义与过程组

[Kerzner 2009]: 项目是具有下列特征的一系列活动和任务:

- 具有一个明确的目标; 有限定的开始和结束日期
- 有成本限制; 消耗人力和非人力资源; 多工种合作

五大过程组与管理活动:

过程组	包含阶段
五大过程组	项目启动 · 项目计划 · 项目执行 · 项目跟踪与控制 · 项目收尾
管理活动	计划制定 · 团队管理 · 成本控制 · 质量保障 · 度量 · 过程管理 · 进度跟踪与控制 · 风险管理 · 配置管理

团队的定义与特征

[Katzenbach 1993]: 团队是为了一致的目的、绩效标准、方法而共担责任并且技能互补的少数人。

高效团队的四个核心特征:

- **共同目标** — 所有成员对目标有清晰、一致的理解
- **共担责任** — 成功与失败由团队共同承担
- **技能互补** — 成员具备不同专业技能，形成完整能力矩阵
- **明确的结构** — 团队内部有清晰的组织形式

💡 **软件项目的特殊性**: 软件开发是以人为主的活动，**人力资源**是软件项目最大的资产

三种团队结构对比

结构	决策方式	优点	缺点	适用场景
主程序员团队	集中，主程序员决定一切	效率高·协调开销少	主程序员瓶颈·成员积极性低	小型精英团队
民主团队	分散，成员共同决策	士气高·创意多	决策慢·职责边界模糊	研究性·探索性项目
开放团队	混合，正式与非正式并存	灵活·平衡	管理复杂·需强沟通	中大型工程项目

💡 现代软件团队的普遍选择：开放团队

典型实例：Scrum 框架——

Product Owner（决策需求优先级） +
Scrum Master（保障流程·移除障碍） +
Dev Team（自组织开发）

团队建设：如何让团队高效运转？

建立团队章程

- 明确角色分工
- 制定行为规范
- 统一决策机制
- 明确沟通渠道

持续积累成功

- 小步快跑
- 及时反馈
- 庆祝里程碑
- 心理安全感

避免“团队杀手”

- 频繁的行政打断
- 不现实的截止日期
- 办公室政治噪音
- 任务切换过于频繁

什么最能激励软件工程师？（Boehm, 1981）

排名	激励因素	管理启示
1	个人发展空间	提供学习机会与技术成长路径
2	技术挑战性	避免将优秀工程师困于重复劳动
3	工作自主性	赋予设计决策权，减少微管理
4	薪酬待遇	重要，但不是驱动优秀工程师的首要因素

不同人员的激励因素对比 [Boehm 1981]

排名	开发人员	项目管理人员	普通人
1	成就感	责任感	成就感
2	发展机遇	成就感	受认可程度
3	工作乐趣	工作乐趣	工作乐趣
4	个人生活	受认可程度	责任感
5	成为技术主管的机会	发展机遇	领先
9	工资	工资	地位

💡 开发人员最在意的是**成就感**和**发展机遇**，工资排名仅第 9 位

避免团队杀手 [DeMarco 1999]

组织和管理团队时要回避下列"团队杀手":

- **防范式管理** — 过度监控, 缺乏信任
- **官僚主义** — 流程大于实效
- **地理分散** — 沟通障碍
- **时间分割** — 成员被多个项目切割
- **产品质量的降低** — 牺牲质量换速度
- **虚假的最后期限** — 不切实际的截止日期
- **小圈子控制** — 少数人垄断决策

风险管理：项目管理的第四维度

风险 = 不确定事件 × 影响程度

项目管理不仅是"铁三角"，风险管理贯穿项目全生命周期



风险类型	示例	应对策略
技术风险	新框架不成熟、性能瓶颈	原型验证 (Spike)、技术预研
人员风险	核心成员离职、技能不足	知识共享、交叉培训 (Bus Factor ≥ 2)
需求风险	需求频繁变更、范围蔓延	迭代交付、变更控制流程
进度风险	低估工作量、依赖延迟	缓冲时间、关键路径分析
外部风险	第三方 API 变更、政策法规	抽象层隔离、合规审查

💡 Knight Capital 案例中，**技术风险**（旧代码未清理）和**进度风险**（赶在开盘前部署）同时爆发

AI 时代的项目管理变革

AI 正在改变项目管理的每个环节

传统做法	AI 时代做法	代表工具/方法
人工估算工作量	AI 辅助估算（基于历史数据）	Linear AI、Jira AI
人工编写项目文档	AI 自动生成 PRD/技术方案初稿	Claude、GPT-4
周会/站会口头同步	AI 自动汇总进度、识别风险	Notion AI、Copilot
经验驱动的风险判断	数据驱动的风险预测模型	ML-based risk scoring
代码评审完全靠人	AI 辅助代码评审 + 人工把关	GitHub Copilot Review

AI 对团队结构的深层影响

传统团队

5-9 人
明确的角色分工
初级→中级→高级
大量重复性编码

AI 增强团队

3-5 人 + AI 助手
角色边界模糊化
"AI-native" 全栈工程师
重心转向设计·审查·决策

关键变化：

- **10× 个体生产力**：AI Coding Agent（Cursor、Claude Code）使单人可完成过去小组的代码量
- **Bus Factor 悖论**：AI 降低了对个人知识的依赖，但增加了对 AI 工具链的依赖
- **新的技能需求**：Prompt Engineering、AI Output Review、系统设计能力变得比编码能力更重要
- **管理者的新角色**：从"分配任务"转向"设计人机协作流程"

💡 **思考**：当 AI 可以写代码时，**软件工程师的核心价值**是什么？

— 是问题定义、架构决策和质量判断

PART 3 · 软件质量保障

软件质量的两个维度

维度	定义	典型度量	Knight Capital 中的体现
符合规格	产品满足需求规格说明书	功能覆盖率·性能指标	新系统"符合规格"——但部署不符合规格
用户满意	用户在使用中感受到价值	NPS·任务完成率	交易对手在 45 分钟内感受到了"不满意"

 质量不只是代码质量——部署过程本身也是质量的一部分
Knight Capital 的代码逻辑是正确的；失效的是发布过程的质量

软件质量保障（SQA）的主要活动

代码评审 (Code Review)

| 人工审查: 逻辑缺陷 · 安全隐患 · 规范违反



静态分析 (Static Analysis)

| 工具自动检测: 潜在缺陷 · 代码规范 · 复杂度



动态测试 (Testing)

| 运行代码验证实际行为: 单元测试 · 集成测试 · 系统测试



过程审计 (Audit)

检查开发与发布过程是否符合既定规范



Knight Capital 最需要的恰恰是这一步

评审 vs 测试的分工: 评审发现"代码写了什么"; 测试发现"代码做了什么"

二者各有盲区, 缺一不可

质量模型与质量属性

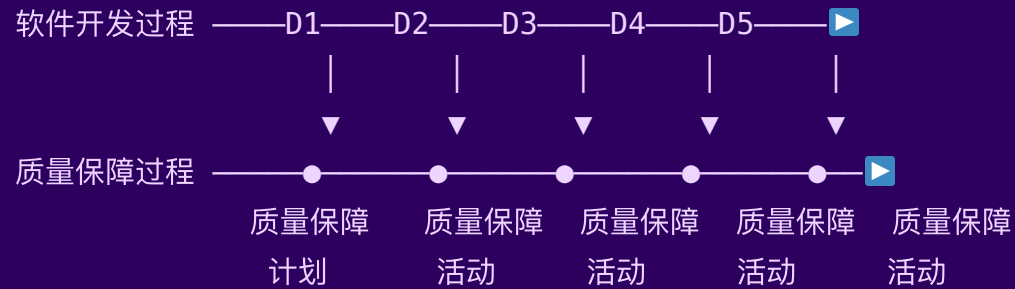
[IEEE 1061-1992, ISO/IEC 9126-1]: 从众多质量属性定义中选择相互配合、相互联系的特征集，称为**质量模型**。

质量因素	说明
功能性	软件提供满足明确和隐含需求的功能的能力
可靠性	软件在指定条件下维持规定性能级别的能力
易用性	软件被用户理解、学习、使用和吸引用户的能力
效率	软件在规定条件下提供适当性能的能力
可维护性	软件被修改的容易程度（修正、改进、适应环境变化）
可移植性	软件从一种环境迁移到另一种环境的能力

质量因素 · 准则 · 度量模型

Use (使用)	Factor (因素)	Criteria (准则)	METRICS
Product operation (产品运行)	—▶ Usability	—▶ Communicativeness	度量指标
	—▶ Reliability	—▶ Accuracy · Consistency	
	—▶ Efficiency	—▶ Device Efficiency · Accessibility	
Product revision (产品修改)	—▶ Reusability	—▶ Completeness · Structuredness	度量指标
	—▶ Maintainability	—▶ Conciseness · Device Independence	
	—▶ Portability	—▶ Legibility · Self-descriptiveness	
	—▶ Testability	—▶ Traceability	

质量保障过程



里程碑	质量保障活动
需求开发	需求评审、需求度量
体系结构	体系结构评审、集成测试（持续集成）
详细设计	详细设计评审、设计度量、集成测试（持续集成）
实现	代码评审、代码度量、测试（测试驱动、持续集成）
测试	测试、测试度量

评审过程

规划 —▶ 总体部署 —▶ 准备 —▶ 审查会议 —▶ 返工 —▶ 跟踪

阶段	英文	关键活动
规划	Planning	制定审查计划，决定次数、时间、地点、人员、内容
总体部署	Overview	描述待审查材料的内容、目标及假设，并分发文档
准备	Preparation	审查人员各自独立执行检查任务，记录问题
审查会议	Inspection Meeting	通过会议讨论，识别、确认、分类错误
返工	Rework	修改发现的缺陷
跟踪	Follow-up	确认所有问题已解决，所有错误已修正

质量度量

"你不能控制自己无法度量的东西" — [DeMarco 1998]

概念	定义	举例
测度 (Measure)	为描述软件产品而提供的定量指标	代码行数
测量 (Measurement)	进行测度的活动	统计所有模块的代码行数
度量 (Metric)	软件产品在特定属性上的量化测度程度	平均代码行数、最大/最小代码行数

AI 时代的软件质量新挑战

当代码不再完全由人编写，质量保障如何演进？

传统质量保障

人写代码 → 人审查代码

缺陷来源：人的疏忽

测试覆盖率是金标准

代码风格人工统一

AI 时代质量保障

AI 生成代码 → 人审查 AI 输出

缺陷来源：幻觉·过时知识·上下文丢失

测试覆盖率 + 语义正确性验证

AI 自动格式化但可能引入隐蔽 Bug

AI 辅助质量保障的实践矩阵

SQA 活动	传统方式	AI 增强方式	注意事项
代码评审	人工逐行审查	AI 预审 + 人工聚焦关键逻辑	AI 可能漏过业务逻辑错误
静态分析	规则引擎 (SonarQube)	LLM 语义分析 + 规则引擎	LLM 结果不确定, 需规则兜底
测试生成	人工编写测试用例	AI 自动生成单元测试	需审查测试是否真正验证了逻辑
缺陷预测	历史缺陷密度统计	ML 模型预测高风险模块	训练数据偏差可能误导优先级
文档审查	人工一致性检查	AI 自动检测文档-代码不一致	减少文档腐化问题

💡 **核心原则:** AI 是质量保障的"加速器"而非"替代者"——
最终的质量判断权必须在人手中

AI 生成代码的质量风险

"AI 写的代码看起来总是对的——这恰恰是最大的风险"

风险类型	表现	应对策略
幻觉代码	调用不存在的 API、编造函数签名	严格编译/测试验证，不信任未验证代码
安全漏洞	训练数据中的不安全模式被复制	安全扫描工具（SAST/DAST）强制检查
版权风险	可能复制开源代码但不遵守许可证	代码溯源工具、许可证合规检查
技术债务	生成"能跑但不好维护"的代码	架构评审、可维护性度量
过度自信	开发者跳过审查"因为是 AI 写的"	建立 AI 代码与人写代码 同等审查标准

PART 4 · 软件配置管理

软件配置管理（SCM）：四大活动



活动	核心问题	Knight Capital 中缺失了什么
配置项确定	哪些制品需要版本控制？	服务器配置状态未纳入管控
配置控制	变更如何审批与执行？	部署无标准流程，无逐台确认
状态报告	各节点版本处于什么状态？	第 8 台服务器状态不可见
配置审计	实际配置是否符合规定？	上线前未做配置一致性审计

配置管理的动机与定义

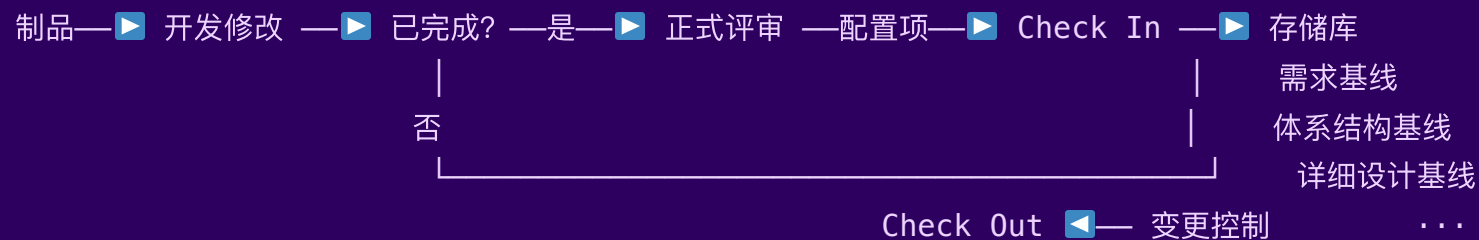
- 软件开发中产生大量**中间制品**（需求规格、设计模型、源码等），是不同阶段协同的基础
- 制品变更的最大挑战：确保使用者获得**最新的制品**，避免协同问题

[IEEE 610.12-1990] 配置管理：用技术的和管理的指导和监督方法，来**标识和说明**配置项的功能和物理特征，**控制**对这些特征的**变更**，**记录和报告**变更处理及其实现状态，并**验证**与规格需求的一致性。

配置项与基线

配置项 (CI): 置于配置管理之下的各种有关项目 (管理文档、评审记录、软件文档、源码、可执行码等)

基线 (Baseline): 已经过正式评审的规格说明或产品, 作为进一步开发的基础, 只有通过正式**变更控制过程**才能变更



版本控制：两种策略

① lock-modify-unlock (锁定-修改-解锁)

Harry LOCK —▶ 编辑 —▶ Write

Sally LOCK x (被拒绝)

Harry UNLOCK

Sally LOCK —▶ 编辑最新版本

✓ 简单安全 ✗ 串行低效

② copy-modify-merge (复制-修改-合并)

Harry & Sally 各自 Copy

—▶ 各自修改 —▶ Sally 先提交

—▶ Harry 得到 "out-of-date"

—▶ Harry merge —▶ 提交合并版

✓ 并行效率高 ✗ 需解决冲突

💡 现代 Git 采用 **copy-modify-merge** 策略，支持分布式并行开发

变更控制：Knight Capital 缺失的那个环节

正确的变更控制流程（Change Control Process）：

变更请求（Change Request）提交



影响分析：哪些模块 / 配置 / 测试用例受影响？



风险评估：是否有不可回滚的操作？ ← Knight Capital 跳过了这一步



CCB（变更控制委员会）评审：批准 / 拒绝 / 制定回滚预案



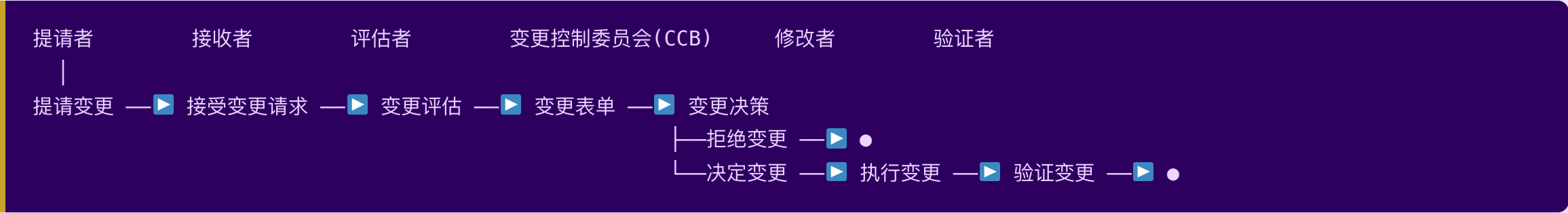
执行变更（逐节点·逐步验证）→ 更新配置基线



上线验证（所有节点一致性检查）→ 更新状态报告

结论：软件部署是一种**配置变更**，必须遵守变更控制流程
"赶进度跳过程序"的代价，可能是公司本身

变更控制的完整流程（6 角色协作）



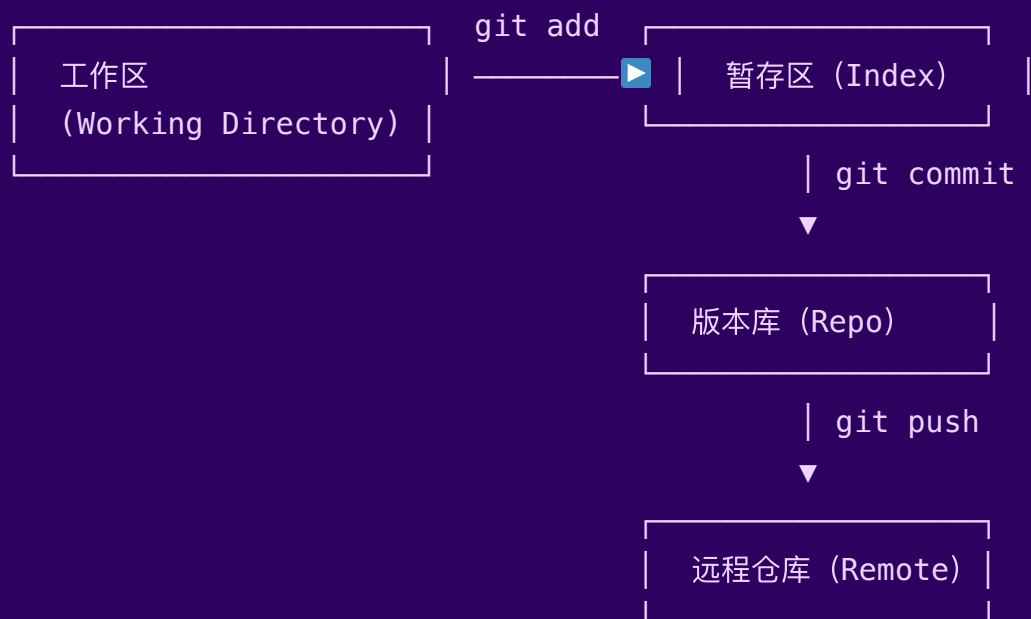
配置管理六大活动：

活动	说明
标识配置项	确定哪些制品需要纳入配置管理
版本管理	跟踪制品的不同版本，确保版本一致性
变更控制	管理变更请求的提出、评估、决策、执行和验证
配置审计	检查配置项是否与规格一致
状态报告	记录和报告配置管理活动的状态
软件发布管理	管理软件产品的发布过程

Git：现代配置管理的基础工具

Git 三大分区与核心操作

开发者本地



命令	比较对象	用途
<code>git diff</code>	工作区 vs 暂存区	查看未 add 的修改
<code>git diff HEAD</code>	工作区 vs 最新提交	查看所有未提交的修改
<code>git diff --cached</code>	暂存区 vs 最新提交	查看已 add 但未 commit 的修改

分支策略：从混乱走向有序

✗ 仅有 main，所有人直接提交：

未成功能污染主线 → 随时可能产生"Knight Capital 1 台旧版本"的问题

✓ 推荐的 Git Flow 简化版：



分支	用途	保护规则
main	生产环境稳定版本	禁止直接 push，只接受合并请求
develop	持续集成分支	合并前必须通过 CI 测试
feature/xxx	功能开发隔离	完成后合并到 develop
hotfix/xxx	生产紧急修复	合并到 main 后同步到 develop

快速实践：识别 SCM 问题（5 分钟）

场景： 你的小组正在开发校园二手交易平台，出现了以下情况：

情况描述	属于哪类 SCM 问题？	用哪个 Git 操作可以预防/解决？
甲同学的代码覆盖了乙同学的修改		
测试环境正常，生产环境崩溃		
不知道上周版本包含哪些功能		
管理员误删了生产配置文件		
需要回滚到两周前的版本		

用 SCM 四大活动（确定·控制·报告·审计）分类，并对应 Git 操作

参考答案：

情况	SCM 问题类型	Git 对应操作
代码覆盖	配置控制缺失	分支隔离 + Pull Request 评审
环境不一致	配置项确定缺失	将配置文件纳入版本控制
版本内容不清晰	状态报告缺失	<code>git log</code> + <code>CHANGELOG.md</code>
误删配置	配置审计缺失	<code>git checkout</code> 恢复历史版本
版本回滚	配置控制	<code>git revert</code> / <code>git reset</code>

从 SCM 到 DevOps：配置管理的现代演进

传统 SCM (2000s)

- | 版本控制
- | 手动构建发布
- | 运维与开发分离
- | 变更走审批流程

DevOps (2010s)

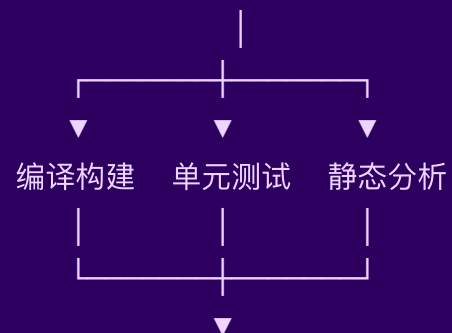
- | CI/CD 自动化管线
- | 基础设施即代码 (IaC)
- | Dev + Ops 融合
- | 自动化测试 + 自动部署

Platform Engineering (2020s)

- | 内部开发者平台 (IDP)
- | 自助式环境创建
- | Golden Path 标准化模板
- | AI 辅助故障诊断

CI/CD: 让"配置一致性"自动化

开发者 push —▶ CI 服务器自动触发



集成测试通过?

├ 否 —▶ 通知开发者修复 (快速反馈)
└ 是 —▶ 自动部署到 Staging

验收测试通过?

├ 否 —▶ 阻断发布
└ 是 —▶ 一键/自动部署到 Production

💡 Knight Capital 教训的现代答案:

CI/CD 管线确保每一次部署都是自动化、可重复、可验证的——
不再依赖人工逐台检查

现代部署策略

策略	原理	优势	适用场景
蓝绿部署	维护两套环境，切换流量	秒级回滚	关键业务系统
金丝雀发布	先向 1–5% 用户发布新版本	早期发现问题，控制爆炸半径	大规模用户产品
滚动更新	逐批替换旧版本实例	零停机	无状态微服务
特性开关	代码已部署但功能未激活	解耦部署与发布	需要灰度验证的功能

⚠ Knight Capital 的"旧 Power Peg 代码"本质上就是一个失控的特性开关——
现代特性开关系统（LaunchDarkly、Unleash）提供统一管理和自动清理

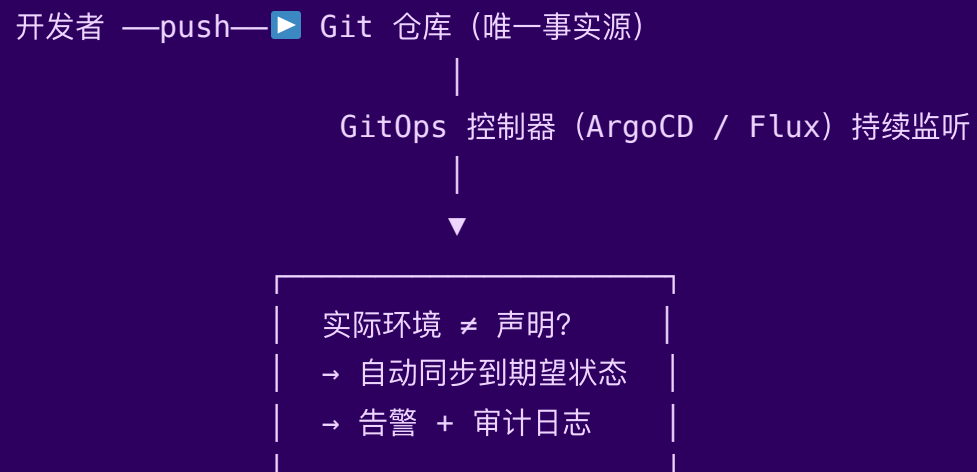
基础设施即代码 (IaC)

IaC: 用代码描述基础设施, 使环境配置可版本化、可审计、可重复

```
# Kubernetes 声明式配置 — 假设 Knight Capital 使用 IaC
apiVersion: apps/v1
kind: Deployment
metadata:
  name: trading-service
spec:
  replicas: 8                # ← 所有 8 台服务器统一定义
  template:
    spec:
      containers:
      - name: trading
        image: trading:v2.1.0  # ← 版本统一, 不可能遗漏 1 台
```

💡 如果 Knight Capital 使用 IaC, 8 台服务器的配置不可能出现不一致——
因为"期望状态"在代码中只定义一次, 由系统自动保证一致

GitOps：以 Git 为中心的运维范式



GitOps 四大原则：

原则	说明	Knight Capital 教训映射
Git 是唯一事实源	所有环境配置都在 Git 仓库中声明	不再有"记不清哪台装了什么"
声明式而非命令式	描述"期望状态"而非"操作步骤"	消除手动操作遗漏
自动同步	系统自动将实际状态与 Git 声明对齐	不一致立即被修复
可审计	<code>git log</code> 即完整变更审计日志	每次变更都有记录 and 责任人

AI 时代的配置管理新范式

领域	变化	代表工具/实践
AI 代码助手的模型版本	模型版本影响生成代码质量，需纳入配置管理	Model cards、版本固定
Prompt 版本管理	Prompt 是新的"源代码"，需版本控制	LangSmith、Prompt Flow
AI Pipeline 配置	RAG 参数、向量库、Embedding 模型构成新配置项	MLflow、Weights & Biases
AI 辅助运维（AIOps）	AI 自动检测异常、预测故障、建议修复	Datadog AI、PagerDuty AI
AI 生成的 IaC	AI 根据需求自动生成基础设施配置	Pulumi AI、Terraform AI

💡 LLMOps = DevOps + ML 模型管理 + Prompt 工程——
这是配置管理在 AI 时代的全新战场

PART 5 · 总结与课后任务

今天建立的知识框架

项目管理基础

— 项目管理

- 铁三角：时间 · 成本 · 范围 → 质量
- 风险管理：识别 · 分析 · 排序 · 应对 · 监控
- 三种团队结构：主程序员 · 民主 · 开放 (Scrum)
- 团队建设：章程 · 持续成功 · 避免团队杀手 · 激励
- 🤖 AI 变革：AI 增强团队 · 角色演变 · 人机协作设计

— 软件质量保障 (SQA)

- 两个维度：符合规格 + 用户满意
- 四类活动：代码评审 · 静态分析 · 测试 · 过程审计
- 质量模型：6 因素 × 因素-准则-度量 体系
- 🤖 AI 变革：AI 辅助审查 · AI 生成代码质量风险 · 人是最终裁判

— 软件配置管理 (SCM)

- 四大活动：确定 · 控制 · 状态报告 · 审计
- 变更控制流程：CR → 影响分析 → CCB → 执行 → 验证
- Git 实践：三分区 · 分支策略 (main/develop/feature/hotfix)
- 现代演进：CI/CD · 部署策略 · IaC · GitOps
- 🤖 AI 变革：LLMOps · Prompt 版本管理 · AIOps

今天最重要的四个结论

1. 团队结构决定协作模式，没有万能答案

选择团队结构时，要考虑**项目性质**（创新 vs 工程）和**团队规模**
Scrum 的开放团队模式适合大多数现代软件项目

2. 质量不只是"代码没 Bug"

SQA 覆盖整个开发与发布过程——**发布过程本身也是质量对象**
Knight Capital 的代码没有 Bug，失效的是发布过程的质量

3. 配置管理是团队协作的工程纪律底线

没有 SCM，团队规模越大，混乱越难控制
1 台服务器的配置不一致 = 4.4 亿美元 + 一家公司

4. AI 改变了效率，但不改变工程纪律的本质

AI 让个体生产力提升 10 倍，但 Knight Capital 式的管理失效**不会因 AI 而消失**
流程·审查·验证的纪律在 AI 时代更加重要，因为**变更速度更快、影响面更大**

从 Knight Capital 到你的课程项目：失效模式完全一样

Knight Capital 失效	校园二手交易平台对应场景
8 台服务器中 1 台未更新	3 台开发机中 1 台依赖库版本不同
旧功能标志未统一清理	废弃的测试代码残留在生产环境
部署无逐台验证 Checklist	发布前未检查所有模块是否编译通过
无自动熔断机制	支付接口异常时整个系统无降级策略

你今天在 Git 实践中建立的分支规范，
就是 Knight Capital 在 2012 年本应拥有的最基础的配置管理实践。

课后作业

个人独立完成

#	任务	要求
1	团队结构分析	描述你课程项目小组的实际协作方式，判断最接近哪种结构，分析其优缺点，100 字
2	Git 分支实操	在本地仓库创建 main / develop / feature 三条分支，完成一次 feature → develop 合并，提交操作日志
3	变更控制设计	为校园二手交易平台设计一份简版"发布前检查清单" (≥ 8 条)，参考 Knight Capital 教训
4	反思文字	150 字：如果 Knight Capital 执行了 SCM 四大活动中的哪一项，事故最有可能被阻止？
5	AI 思考（加分）	200 字：选择项目管理/SQA/SCM 中的一个领域，分析 AI 工具如何改变该领域的实践，并指出 AI 不能替代的部分

- 💡 第 3 题：检查清单就是 SCM"配置审计"活动的实际形态
- 💡 第 5 题：关键在于区分"效率提升"与"判断替代"——AI 加速流程，但工程纪律的决策权在人

想深入了解？推荐阅读

级别	材料	核心洞察
● 必读	SEC: In the Matter of Knight Capital Americas LLC (2013)	原始监管调查报告，SCM 失效的第一手分析
● 必读	How to Lose \$172,222 a Second for 45 Minutes (Doug Seven, 2013)	工程师视角还原 Knight Capital 事故全过程
● 推荐	Pro Git (Scott Chacon, 免费在线) 第 3 章：Git 分支	Git 分支模型的权威入门
● 推荐	A Successful Git Branching Model (Vincent Driessen, nvie.com)	Git Flow 原始提案，仍是工业界参考
● 扩展	DeMarco & Lister: Peopleware (1987, 第 3 版 2013)	团队建设与组织环境的经典著作
● AI	Large Language Models for Software Engineering: A Systematic Literature Review (2024)	LLM 在 SE 各环节的应用全景综述
● AI	Google: DORA State of DevOps Report (2024)	AI 对 DevOps 效能的最新实证数据
● AI	Measuring Developer Productivity with AI (GitHub, 2024)	AI Coding 工具的生产力度量方法论

下节课：模块二 · 单元 1

需求工程基础

- 需求工程四个核心活动：获取 · 分析 · 规格说明 · 验证
- 什么样的需求才是"好需求"? INVEST 准则
- 用户故事标准格式：角色 · 功能 · 价值 · 验收标准
- 实践：为校园二手交易平台编写第一批用户故事

课前思考：

如果让你用一句话描述校园二手交易平台的核心需求，
你会怎么写？

这句话里有没有**模糊词**（"方便""快速""及时"）？

本节课结束

"配置管理是工程纪律，不是可选项——Knight Capital 用 4.4 亿美元证明了这一点。"

"AI 让我们写代码更快，但不能让我们跳过验证——速度越快，纪律越重要。"

"团队的混乱大多不是因为技术太难，而是因为过程没有被认真对待。"

软件工程与计算 II · 模块一 · 单元 2